

Lambda Calculus

What is the simplest formalism for functions (viewed as rules to compute something)?

Lambda calculus has:

- ▶ function application: fx (means f applies to x , what we usually write $f(x)$)
- ▶ a way to construct a function $\lambda x \mapsto e$, in math $x \mapsto e$, traditionally $\lambda x.e$

Evaluation rule (β reduction): substitute argument into function body.

Math $(x \mapsto x^2)(x+1)$ evaluates to $(x+1)^2$

Lambda calculus $(\lambda x.x^2)(x+1)$ $\longrightarrow^\beta$ $(x+1)^2$

λ calculus is Turing-complete: an encoding of natural numbers (Church numerals), an encoding of general recursion (Y combinator), a way to represent all Turing machines.

Due to non-termination, we use typed versions, which have natural models in classical set theory. Simplest: classical higher-order logic (Church's simple theory of types)

The Seven Virtues of Simple Type Theory by William Farmer

<https://imps.mcmaster.ca/doc/seven-virtues.pdf>

A1 (Truth Values)

$$\forall f : (* \rightarrow *) . (f(\top) \wedge f(\bot)) \Leftrightarrow (\forall x : * . f(x)).$$

A2 (Leibniz' Law⁶)

$$\forall x, y : \alpha . (x = y) \Rightarrow (\forall p : (\alpha \rightarrow *) . p(x) \Leftrightarrow p(y)).$$

A3 (Extensionality)

$$\forall f, g : (\alpha \rightarrow \beta) . (f = g) \Leftrightarrow (\forall x : \alpha . f(x) = g(x)).$$

A4 (Beta-Reduction)

$$(\lambda x : \alpha . B_\beta)(A_\alpha) = B_\beta[(x : \alpha) \mapsto A_\alpha]$$

provided A_α is free for $(x : \alpha)$ in B_β .⁷

A5 (Proper Definite Description)

$$(\exists ! x : \alpha . A_*) \Rightarrow A_*[(x : \alpha) \mapsto (Ix : \alpha . A_*)]$$

provided $(Ix : \alpha . A_*)$ is free for $(x : \alpha)$ in A_* .⁷

A6 (Improper Definite Description)

$$\neg(\exists ! x : \alpha . A_*) \Rightarrow (Ix : \alpha . A_*) = \perp_\alpha.$$

R (Equality Substitution) From $A_\alpha = B_\alpha$ and C_* infer the result of replacing one occurrence of A_α in C_* by an occurrence of B_α , provided that the occurrence of A_α in C_* is not a variable $(x : \alpha)$ immediately preceded by λ or I .

Systems: *HOL4, Isabelle*

book: *P. Andrews: To Truth through Proof*

Natural Deduction Proofs

Modus ponens is easy and natural rule to use: if we proved $p \rightarrow q$ and also proved p , then we can prove q .

$$\frac{p \rightarrow q \quad p}{q}$$

We want a dual *deduction* rule: to prove $p \rightarrow q$, just *temporarily assume* p and then prove q . Afterwards conclude $p \rightarrow q$ and stop assuming p .

Temporary assumption is in some semi-pictorial notations written $[p]$, so we have rule:

$$\frac{\begin{array}{c} [p] \\ \dots \\ q \end{array}}{p \rightarrow q}$$

We will instead introduce a context, Γ that stores our assumptions and use *sequents*.

Sequents: Describing Valid Deductions from Assumptions

Let F be a formula and Γ a set of formulas.

Notation $\Gamma \vdash F$ denotes a new kind of formula, called a *sequent*. In general a sequent $F_1, \dots, F_n \vdash G_1, \dots, G_m$ has the same semantics as $(F_1 \wedge \dots \wedge F_n) \longrightarrow (G_1 \vee \dots \vee G_m)$.

Now, we can formulate how to prove the deduction rule like this:

$$\frac{\Gamma \cup \{p\} \vdash q}{\Gamma \vdash p \rightarrow q}$$

Modus ponens looks similar as before, just with “ $\Gamma \vdash$ ” added:

$$\frac{\Gamma \vdash p \rightarrow q \quad \Gamma \vdash p}{\Gamma \vdash q}$$

Of course, we will also have the proof by assumption, which is this axiom schema:

$$\Gamma \cup \{p\} \vdash p$$

Exercises 1: Prove Using Sequents

$a \rightarrow a$: In a previous set of rules and axioms, the proof was difficult to find. Now, we have this proof:

$$\frac{a \vdash a \quad (\text{assumption})}{\emptyset \vdash a \rightarrow a \quad (\text{deduction})}$$

$a \rightarrow (b \rightarrow a)$: In some systems, it's an axiom. We now have:

$$\frac{\frac{\{a, b\} \vdash a \quad (\text{assumption})}{\{a\} \vdash b \rightarrow a \quad (\text{deduction})}}{\emptyset \vdash a \rightarrow (b \rightarrow a) \quad (\text{deduction})}$$

Hint: These proofs are easier to construct backwards! That's what tactics like **intros** help us do.

Exercises 2: Prove Using Sequents

$$(a \rightarrow (b \rightarrow c)) \rightarrow ((a \rightarrow b) \rightarrow (a \rightarrow c)):$$

Exercises 2: Prove Using Sequents

$(a \rightarrow (b \rightarrow c)) \rightarrow ((a \rightarrow b) \rightarrow (a \rightarrow c))$: The second part of the proof is:

$$\frac{\frac{\frac{\{a \rightarrow (b \rightarrow c), a \rightarrow b, a\} \vdash c}{\{a \rightarrow (b \rightarrow c), a \rightarrow b\} \vdash a \rightarrow c}}{a \rightarrow (b \rightarrow c) \vdash (a \rightarrow b) \rightarrow (a \rightarrow c)}}{\emptyset \vdash (a \rightarrow (b \rightarrow c)) \rightarrow ((a \rightarrow b) \rightarrow (a \rightarrow c))}$$

Let Γ be $\{a \rightarrow (b \rightarrow c), a \rightarrow b, a\}$. Then:

1. $\Gamma \vdash a$ (assumption)
2. $\Gamma \vdash a \rightarrow b$ (assumption)
3. $\Gamma \vdash b$ (modus ponens from 2 and 1)
4. $\Gamma \vdash a \rightarrow (b \rightarrow c)$ (assumption)
5. $\Gamma \vdash b \rightarrow c$ (modus ponens from 4 and 1)
6. $\Gamma \vdash c$ (modus ponens from 5 and 3)

and this is the first part.

Types for Lambda Calculus

If $f: A \rightarrow B$ and $g: B \rightarrow C$, note that their composition is $\lambda x: A \mapsto g(fx)$ and that it maps $A \rightarrow C$, so we can write:

$$\lambda x: A \mapsto g(fx) : A \rightarrow C$$

We will now define rules that determine the domains and codomains of functions in lambda calculus, under given assumptions. We will also use “ $\Gamma \vdash$ ” to describe such rules. In this example, we will have:

$$\{f: A \rightarrow B, g: B \rightarrow C\} \vdash \lambda x: A \mapsto g(fx) : A \rightarrow C$$

Because we can have functions that return other functions, we can have nested function types, like an operator on real functions, $(R \rightarrow R) \rightarrow (R \rightarrow R)$. The set to which functions belong will be called a type, so $f: A \rightarrow B$ is a special case of saying that function f has type $A \rightarrow B$.

In general $\Gamma \vdash t: F$ means: if we assume the types of variables according to Γ , then the expression t has type F .

Rules for Simply Typed Lambda Calculus

Application:

$$\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash x : A}{\Gamma \vdash (fx) : B}$$

Abstraction (for x not in Γ):

$$\frac{\Gamma \cup \{x : A\} \vdash t : B}{\Gamma \vdash (\lambda x : A \mapsto t) : (A \rightarrow B)}$$

Variable:

$$\Gamma \cup \{x : A\} \vdash x : A$$

Note: today we use notation $\lambda x:A \mapsto t$ (symbol “ $\mapsto$ ” is “ $\backslashmapsto$ ”) for a function that maps x to t . Lean also accepts **fun** instead of λ , as well as $\Rightarrow$ instead of $\mapsto$.

Example of Typing

1. $\{f: A \rightarrow B, g: B \rightarrow C, x: A\} \vdash f : A \rightarrow B$ (variable)
2. $\{f: A \rightarrow B, g: B \rightarrow C, x: A\} \vdash x : A$ (variable)
3. $\{f: A \rightarrow B, g: B \rightarrow C, x: A\} \vdash fx : B$ (application 1, 2)
4. $\{f: A \rightarrow B, g: B \rightarrow C, x: A\} \vdash g : B \rightarrow C$ (variable)
5. $\{f: A \rightarrow B, g: B \rightarrow C, x: A\} \vdash g(fx) : C$ (application 4, 3)
6. $\{f: A \rightarrow B, g: B \rightarrow C\} \vdash \lambda x:A \mapsto g(fx) : A \rightarrow C$ (abstraction 5)

Correspondence (Curry-Howard, Propositions as Types)

lambda calculus

$$\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash x : A}{\Gamma \vdash (fx) : B}$$

$$\frac{\Gamma \cup \{x : A\} \vdash t : B}{\Gamma \vdash (\lambda x : A. t) : (A \rightarrow B)}$$

$$\Gamma \cup \{x : A\} \vdash x : A$$

sequents

$$\frac{\Gamma \vdash A \rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B}$$

$$\frac{\Gamma \cup \{A\} \vdash B}{\Gamma \vdash A \rightarrow B}$$

$$\Gamma \cup \{A\} \vdash A$$

We can treat:

- ▶ types as propositions (true or false)
- ▶ lambda expression of t type F as proof of F

Note that a proof that locally assumes A and continues as t is the function $\lambda x : A. t$.

Example

We had this proof for $a \rightarrow (b \rightarrow a)$ using sequents:

$$\frac{\frac{\frac{\{a, b\} \vdash a \quad (\text{assumption})}{\{a\} \vdash b \rightarrow a \quad (\text{deduction})}}{\emptyset \vdash a \rightarrow (b \rightarrow a) \quad (\text{deduction})}}$$

Equivalently, find a lambda expression of type $a \rightarrow (b \rightarrow a)$. What is it?

Example

We had this proof for $a \rightarrow (b \rightarrow a)$ using sequents:

$$\frac{\frac{\frac{\{a, b\} \vdash a \quad (\text{assumption})}{\{a\} \vdash b \rightarrow a \quad (\text{deduction})}}{\emptyset \vdash a \rightarrow (b \rightarrow a) \quad (\text{deduction})}}$$

Equivalently, find a lambda expression of type $a \rightarrow (b \rightarrow a)$. What is it?

$$\lambda x:a \mapsto (\lambda y:b \mapsto x)$$

By adding terms to sequent proof above, we obtain an evidence that the function above indeed has the type $a \rightarrow (b \rightarrow a)$.

False and Negation

We introduce False. It is the thing from which everything follows.
So, assume that, for every P , we have a constant

$$\text{FalseElim} : \text{False} \rightarrow P$$

Moreover, we will define $\neg P$ as a shorthand for $P \rightarrow \text{False}$.

Then, to prove $\Gamma \vdash \neg P$, we show that there is a proof c where $\Gamma \cup \{h : P\} \vdash c : \text{False}$.

Then, we have

1. $\Gamma \cup \{h:P\} \vdash c : \text{False}$ (assumed starting point)
2. $\Gamma \vdash (\lambda h:P \mapsto c) : P \rightarrow \text{False}$ (abstraction)
3. $\Gamma \vdash (\lambda h:P \mapsto c) : \neg P$ (definition of negation)

In other words, this definition of False gives us one variant of proof by contradiction.

Exercise: Double Negation Introduction

Prove $\boxed{P \rightarrow \neg\neg P}$. Recall $\neg P = P \rightarrow \text{False}$, so $\neg\neg P$ is $(P \rightarrow \text{False}) \rightarrow \text{False}$.

Let $\Gamma = \{h : P, k : \neg P\}$.

1. $\Gamma \vdash h : P$ (variable)
2. $\Gamma \vdash k : P \rightarrow \text{False}$ (variable)
3. $\Gamma \vdash kh : \text{False}$ (application 2, 1)
4. $\{h:P\} \vdash (\lambda k:(P \rightarrow \text{False}) \mapsto kh) : (P \rightarrow \text{False}) \rightarrow \text{False}$ (abstraction 3)
5. $\emptyset \vdash (\lambda h:P \mapsto \lambda k:(P \rightarrow \text{False}) \mapsto kh) : P \rightarrow (P \rightarrow \text{False}) \rightarrow \text{False}$
(abstraction 4)

Exercise (in Futility): Double Negation Elimination

Using the rules we introduced so far, prove $(\neg\neg P) \rightarrow P$.

It cannot be done. We need an axiom (an alternative is to become an intuitionists):

$$\text{@Classical.byContradiction} : ((P \rightarrow \text{False}) \rightarrow \text{False}) \rightarrow P$$

So, propositions as types works for intuitionistic logic. We want classical logic, so from time to time we invoke a specific small set of classical axioms.

Rules for Conjunction

Deduction rules:

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B}$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A}$$

$$\frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B}$$

Lambda calculus (treat $A \wedge B$ as $A \times B$):

$$\frac{\Gamma \vdash x : A \quad \Gamma \vdash y : B}{\Gamma \vdash (x, y) : A \wedge B}$$

$$\frac{\Gamma \vdash p : A \wedge B}{\Gamma \vdash p.1 : A}$$

$$\frac{\Gamma \vdash p : A \wedge B}{\Gamma \vdash p.2 : B}$$

A proof for $A \wedge B$ is a pair of a proof for A and a proof for B

Notation: $(x, y).1 = x$ and $(x, y).2 = y$

To avoid confusion with other pairs, Lean uses $(\text{And.intro } x y)$ instead of directly (x, y) as well as $p.\text{left}$ instead of $p.1$ and $p.\text{right}$ instead of $p.2$.

Beyond Propositional Logic: Quantifiers

Universal Quantifier Deduction Rules

(We use the French comma after quantified variable, as in Lean.)

Eliminating (instantiating) quantifier ($x := t$ means replace x with t):

$$\Gamma \cup \{\forall x:I, P\} \vdash P[x := t]$$

Introducing quantifier (when $x \notin \Gamma$):

$$\frac{\Gamma \vdash P}{\Gamma \vdash \forall x:I, P}$$

Previously $I \rightarrow Q$ is type of functions from I to Q , if we have proposition I we could conclude Q .

Now, our types have variables inside, like P , so given $t : I$ we can conclude $P[x := t]$.

To represent $P(x)$ we will have a family of types indexed by I .

We will model the universally quantified type using Cartesian product of the family.

Product of a Family in Set Theory

Consider a family P of sets indexed by I .

For each $i \in I$, we have that $P(i)$ is a set, so P is a function from I to a set of sets.

A function is *f conforms to P* iff, for every $i \in I$, the value $f(i)$ belongs to $P(i)$.

We define the product of P as the space of all functions that conform to P :

$$\prod P := \prod_{i \in I} P(i) := \{f: I \rightarrow U \mid f \text{ conforms to } P\}$$

Above, we can set $U := \bigcup_{i \in I} P(i)$

Example. We can represent $A \times B$ by taking $I = \{1, 2\}$ and $P(1) = A$, $P(2) = B$. An element of $\prod P$ is a function mapping 1 to an element of A and 2 to an element of B , which corresponds to a pair $(f(1), f(2)) \in A \times B$.

Summary: Product of a Family of sets P

$$\prod_{i \in I} P(i) := \{f: I \rightarrow \bigcup_{i \in I} P(i) \mid \forall x \in I. f(x) \in P(x)\}$$

Rules for Typed Lambda Calculus

We had these two rules (simple types):

Application (Simple):

$$\frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash x : A}{\Gamma \vdash (fx) : B}$$

Abstraction (simple, for x not in Γ and not possible in B but in t):

$$\frac{\Gamma \cup \{x : A\} \vdash t : B}{\Gamma \vdash (\lambda x : A \mapsto t) : (A \rightarrow B)}$$

Now we have more general rules (dependent types):

Application (Dependent, when $P = (\lambda x \mapsto B)$ we get the old rule):

$$\frac{\Gamma \vdash f : \prod P \quad \Gamma \vdash x : A}{\Gamma \vdash (fx) : (Px)}$$

Abstraction (dependent, for x not in Γ but possibly in both t and B):

$$\frac{\Gamma \cup \{x : A\} \vdash t : B}{\Gamma \vdash (\lambda x : A \mapsto t) : \prod (\lambda x : A \mapsto B)}$$

Correspondence with Dependent Types

Deduction rules

Dependently Typed Lambda Calculus

Instantiation ($\forall$ -elimination):

Application:

$$\frac{\Gamma \vdash \forall x, P(x)}{\Gamma \vdash P(t)}$$

$$\frac{\Gamma \vdash f : \prod P \quad \Gamma \vdash t : A}{\Gamma \vdash (ft) : (Pt)}$$

Generalization ($\forall$ -introduction):

Abstraction:

$$\frac{\Gamma \vdash B}{\Gamma \vdash \forall x, B}$$

$$\frac{\Gamma \cup \{x : A\} \vdash t : B}{\Gamma \vdash (\lambda x:A \mapsto t) : \prod (\lambda x:A \mapsto B)}$$

We use $\prod (\lambda x:A \mapsto B)$ type to represent $\forall x, B$.

Dependent Pair Corresponds to Existentials

$$\sum_{i \in I} P(i) := \{ (i, x) \mid x \in P(i) \}$$

Deduction rules

Using a witness ($\exists$ -introduction):

$$\frac{\Gamma \vdash i : I \quad \Gamma \cup \{i : I\} \vdash P(i)}{\Gamma \vdash \exists i : I, P(i)}$$

Picking a fresh witness
($\exists$ -elimination):

$$\frac{\Gamma \vdash \exists i : I, P(i)}{\Gamma \vdash P(i) \quad (i \notin FV(\Gamma))}$$

Dependently Typed Lambda Calculus

Construction:

$$\frac{\Gamma \vdash i : I \quad \Gamma \vdash x : (P i)}{\Gamma \vdash (i, x) : \sum_{i \in I} (P i)}$$

Deconstruction:

$$\frac{\Gamma \vdash (i, x) : \sum_{i \in I} (P i)}{\Gamma \vdash i : I}$$

$$\frac{\Gamma \vdash (i, x) : \sum_{i \in I} (P i)}{\Gamma \vdash x : (P i)}$$